

BÁO CÁO BÀI TẬP VỀ NHÀ

CHỦ ĐỀ: ĐA NĂNG HÓA TOÁN

TỬ IN C++



Thực hiện: Trần Thịnh Phát

Lớp: 25T_KHDL

Môn học: Lập trình hướng đối tượng

Hạn nộp: 25/09/2026

Mục lục

Bài 1: Lớp Đa thức (Polynomial)	2
1.1 Mô tả các thành viên của lớp	2
1.1.1 Thành phần dữ liệu (Private Attributes)	2
1.1.2 Danh sách các hàm và toán tử (Public Methods & Operators)	3
1.2 Cơ sở Giải thuật	3
1.2.1 Biểu diễn Đa thức và Quản lý Bộ nhớ	3
1.2.2 Cộng và Trừ hai Đa thức (+, -)	3
1.2.3 Tính giá trị Đa thức (operator())	4
1.2.4 Định dạng Xuất Đa thức (operator«)	4
1.3 Mã nguồn chương trình	4
1.4 Kết quả chạy thử nghiệm	7
Bài 2: Lớp Tập hợp các số nguyên (SET)	8
2.1 Mô tả các thành viên của lớp	8
2.1.1 Thành phần dữ liệu (Private Attributes)	8
2.1.2 Danh sách các hàm và toán tử (Public Methods & Operators)	9
2.2 Cơ sở Giải thuật	9
2.2.1 Cách quản lý mảng và Thêm số mới (insert)	9
2.2.2 Giải thuật cho Hợp, Giao, Hiệu	9
2.2.3 Giải thuật Nhập dữ liệu an toàn (»)	10
2.3 Mã nguồn chương trình	10
2.4 Kết quả chạy thử nghiệm	14
Bài 3: Lớp Ma trận vuông (Matrix)	15
3.1 Mô tả các thành viên của lớp	15
3.1.1 Thành phần dữ liệu và Hàm nội bộ (Private)	15
3.1.2 Danh sách các hàm và toán tử (Public Methods & Operators)	16
3.2 Cơ sở Giải thuật	16
3.2.1 Sử dụng mảng 1 chiều để biểu diễn ma trận	16
3.2.2 Quản lý bộ nhớ tập trung	16
3.2.3 Cài đặt các phép toán (+, -, *, +=)	17
3.3 Mã nguồn chương trình	17
3.4 Kết quả chạy thử nghiệm	21

1 Bài 1: Lớp Đa thức (Polynomial)

Yêu cầu đề bài

Cài đặt lớp Đa thức `Polynomial` để biểu diễn 01 đa thức bậc n với 1 biến x , bao gồm các hàm thành viên:

- Hàm dựng, hàm hủy, hàm dựng sao chép.
- Toán tử $+$, $-$ hai đa thức.
- Toán tử $\gg$, $\ll$ để nhập, xuất đa thức.
- Toán tử $=$ để gán hai đa thức.
- Toán tử $[]$ để truy xuất một hệ số của đa thức.
- Toán tử $()$ để tính giá trị của đa thức.

1.1 Mô tả các thành viên của lớp

1.1.1 Thành phần dữ liệu (Private Attributes)

- `int deg`: Lưu bậc cao nhất n của đa thức ($n \geq 0$).
- `double *coeffs`: Con trỏ quản lý mảng động chứa các hệ số. Mảng có kích thước $n + 1$, trong đó `coeffs[i]` lưu hệ số đi kèm với x^i .

1.1.2 Danh sách các hàm và toán tử (Public Methods & Operators)

Phương thức / Toán tử	Ý nghĩa và chức năng
Polynomial(int d = 0)	Hàm dựng khởi tạo đa thức bậc d , cấp phát $d + 1$ phần tử và gán bằng 0.
Polynomial(const Polynomial&)	Hàm dựng sao chép (Deep Copy) để tạo bản sao độc lập dữ liệu.
~Polynomial()	Hàm hủy, giải phóng mảng động <code>coeffs</code> tránh rò rỉ bộ nhớ.
operator=	Toán tử gán hai đa thức, có kiểm tra tự gán và giải phóng bộ nhớ cũ.
operator+	Toán tử cộng hai đa thức, trả về một đối tượng đa thức tổng mới.
operator-	Toán tử trừ hai đa thức, trả về đối tượng đa thức hiệu mới.
operator[]	Truy xuất hoặc thay đổi hệ số tại vị trí bậc <code>idx</code> bằng tham chiếu.
operator()	Yêu cầu nhập giá trị x từ bàn phím và tính giá trị của đa thức $P(x)$.
operator»	Toán tử nhập bậc và từng hệ số từ x^n xuống x^0 .
operator«	Toán tử xuất đa thức ra màn hình dưới dạng chuẩn.

Bảng 1: Bảng tổng hợp các hàm và toán tử của lớp Polynomial

1.2 Cơ sở Giải thuật

1.2.1 Biểu diễn Đa thức và Quản lý Bộ nhớ

- Lưu trữ dữ liệu: Chương trình cấp phát mảng động `coeffs` gồm $n + 1$ phần tử. Chỉ số của mảng được thiết kế trùng khớp trực tiếp với số mũ của x (ví dụ: `coeffs[i]` lưu hệ số của x^i).
- Sao chép sâu (Deep Copy): Trong hàm dựng sao chép và toán tử gán `=`, chương trình cấp phát vùng nhớ mới rồi mới chép dữ liệu sang, giúp tránh lỗi hai đối tượng trỏ chung vùng nhớ (lỗi Double Free khi giải phóng).

1.2.2 Cộng và Trừ hai Đa thức (+, -)

1. Tìm bậc lớn nhất giữa hai đa thức: `maxDeg = max(deg, other.deg)`.
2. Khởi tạo đa thức kết quả có bậc bằng `maxDeg`.
3. Duyệt chỉ số i từ 0 đến `maxDeg`.

4. Sử dụng toán tử ba ngôi để lấy hệ số an toàn (nếu chỉ số i vượt quá bậc của đa thức nào thì lấy giá trị 0):

```
double a = (i <= deg) ? coeffs[i] : 0;  
double b = (i <= other.deg) ? other.coeffs[i] : 0;
```

5. Gán hệ số mới cho đa thức kết quả bằng tổng hoặc hiệu: `result.coeffs[i] = a + b` (hoặc `a - b`).

1.2.3 Tính giá trị Đa thức (operator())

1. Nhận giá trị x nhập vào từ bàn phím.
2. Khởi tạo biến tích lũy `result = 0`.
3. Chạy vòng lặp từ $i = 0$ đến bậc n , tính toán và cộng dồn: `result += coeffs[i] * pow(x, i)`.

1.2.4 Định dạng Xuất Đa thức (operator«)

1. Khởi tạo cờ hiệu `bool f = true` để đánh dấu số hạng đầu tiên xuất ra.
2. Duyệt chỉ số i giảm dần từ bậc cao nhất n về 0:
 - Nếu hệ số `coeffs[i] == 0`: Bỏ qua không in.
 - Nếu hệ số dương và không phải số hạng đầu tiên (`!f`): In thêm dấu `" + "`.
 - Nếu hệ số âm: In dấu `" - "`.
 - In giá trị tuyệt đối của hệ số bằng `abs(coeffs[i])`.
 - Nếu $i > 0$: In phần biến `"x^i"`.
 - Đặt lại cờ `f = false`.
3. Trường hợp đặc biệt: Nếu tất cả hệ số đều bằng 0 (cờ `f` vẫn là `true`), xuất ra màn hình ký tự `"0"`.

1.3 Mã nguồn chương trình

File Mã Nguồn: `Polynomial.cpp`

```
1 #include <iostream>  
2 #include <cmath>  
3 using namespace std;  
4
```

```
5 class Polynomial {
6 private:
7     int deg;
8     double *coeffs;
9
10 public:
11     Polynomial(int d = 0) {
12         deg = d;
13         coeffs = new double[deg + 1];
14         for (int i = 0; i <= deg; i++) coeffs[i] = 0;
15     }
16     Polynomial(const Polynomial& other) {
17         deg = other.deg;
18         coeffs = new double[deg + 1];
19         for (int i = 0; i <= deg; i++) coeffs[i] = other.coeffs[i];
20     }
21     ~Polynomial() {
22         delete[] coeffs;
23     }
24     Polynomial& operator=(const Polynomial& other) {
25         if (this != &other) {
26             delete[] coeffs;
27             deg = other.deg;
28             coeffs = new double[deg + 1];
29             for (int i = 0; i <= deg; i++) coeffs[i] = other.coeffs[i];
30         }
31         return *this;
32     }
33     Polynomial operator+(const Polynomial& other) const {
34         int maxDeg = max(deg, other.deg);
35         Polynomial result(maxDeg);
36         for (int i = 0; i <= maxDeg; i++) {
37             double a = (i <= deg) ? coeffs[i] : 0;
38             double b = (i <= other.deg) ? other.coeffs[i] : 0;
39             result.coeffs[i] = a + b;
40         }
41         return result;
42     }
43     Polynomial operator-(const Polynomial& other) const {
44         int maxDeg = max(deg, other.deg);
```

```
45     Polynomial result(maxDeg);
46     for (int i = 0; i <= maxDeg; i++) {
47         double a = (i <= deg) ? coeffs[i] : 0;
48         double b = (i <= other.deg) ? other.coeffs[i] : 0;
49         result.coeffs[i] = a - b;
50     }
51     return result;
52 }
53 double& operator[](int idx) {
54     return coeffs[idx];
55 }
56 double operator()() const {
57     double x;
58     cout << "Nhap gia tri x tu ban phim: ";
59     cin >> x;
60     double result = 0;
61     for (int i = 0; i <= deg; i++){
62         result += coeffs[i]*pow(x,i);
63     }
64     return result;
65 }
66 friend istream& operator>>(istream& in, Polynomial& p) {
67     cout << "Nhap bac cua da thuc: ";
68     in >> p.deg;
69     delete[] p.coeffs;
70     p.coeffs = new double[p.deg + 1];
71     for (int i = p.deg; i >= 0; i--) {
72         cout << "He so cua x^" << i << " la: ";
73         in >> p.coeffs[i];
74     }
75     return in;
76 }
77 friend ostream& operator<<(ostream& out, const Polynomial& p) {
78     bool f = true;
79     for (int i = p.deg; i >= 0; i--) {
80         if (p.coeffs[i] != 0) {
81             if (!f && p.coeffs[i] > 0) out << " + ";
82             if (p.coeffs[i] < 0) out << " - ";
83             out << abs(p.coeffs[i]);
84             if (i > 0) out << "x^" << i;
```

```

85         f = false;
86     }
87 }
88     if (f) out << "0";
89     return out;
90 }
91 };
92 int main() {
93     Polynomial p1, p2;
94     cout << "--- Nhập đa thức P1 ---\n";
95     cin >> p1;
96     cout << "--- Nhập đa thức P2 ---\n";
97     cin >> p2;
98     cout << "\nP1(x) = " << p1 << "\n";
99     cout << "P2(x) = " << p2 << "\n";
100    Polynomial p3 = p1 + p2;
101    cout << "P1 + P2 = " << p3 << "\n";
102    Polynomial p4 = p1 - p2;
103    cout << "P1 - P2 = " << p4 << "\n";
104    cout << "\n--- Tính giá trị P1 với x nhập từ bàn phím ---\n";
105    cout << p1() << "\n";
106    return 0;
107 }

```

1.4 Kết quả chạy thử nghiệm

Chương trình được biên dịch và chạy thử nghiệm trên môi trường C++ với các tham số mẫu:

- Đa thức $P_1(x) = 3x^2 - 2x + 1$ (Bậc 2).
- Đa thức $P_2(x) = 5x^2 + 4x - 3$ (Bậc 2).
- Kiểm tra tính giá trị $P_1(x)$ tại $x = 2$.

Kết quả xuất ra màn hình (Console Output)

```

--- Nhập đa thức P1 ---
Nhập bậc của đa thức: 2
He so cua x^2 la: 3
He so cua x^1 la: -2
He so cua x^0 la: 1

```

```
--- Nhập đa thức P2 ---
Nhập bậc của đa thức: 2
He so cua x^2 la: 5
He so cua x^1 la: 4
He so cua x^0 la: -3

P1(x) = 3x^2 - 2x^1 + 1
P2(x) = 5x^2 + 4x^1 - 3
P1 + P2 = 8x^2 + 2x^1 - 2
P1 - P2 = - 2x^2 - 6x^1 + 4

--- Tính giá trị P1 với x nhập từ bàn phím ---
Nhập giá trị x từ bàn phím: 2
9
```

2 Bài 2: Lớp Tập hợp các số nguyên (SET)

Yêu cầu đề bài

Cài đặt lớp tập hợp các số nguyên **SET** gồm số phần tử hiện có, **data** (mảng động), **size** (kích thước). Định nghĩa các hàm và toán tử:

- Hàm dựng mặc định, hàm dựng sao chép, hàm hủy.
- Các hàm hỗ trợ: kiểm tra phần tử tồn tại, thêm phần tử.
- Toán tử $+$, $*$, $-$ giữa hai tập hợp (Hợp, Giao, Hiệu).
- Toán tử $[]$ để lấy phần tử theo vị trí.
- Toán tử so sánh $==$ giữa hai tập hợp.
- Toán tử $\gg$, $\ll$ để nhập và in tập hợp.

2.1 Mô tả các thành viên của lớp

2.1.1 Thành phần dữ liệu (Private Attributes)

- `int size`: Số lượng phần tử hiện đang có trong tập hợp.
- `int *data`: Con trỏ dùng để tạo mảng động chứa các con số.

2.1.2 Danh sách các hàm và toán tử (Public Methods & Operators)

Phương thức / Toán tử	Chức năng
<code>Set(int s = 0)</code>	Tạo tập hợp, cấp phát mảng với kích thước ban đầu.
<code>Set(const Set&)</code>	Chép toàn bộ dữ liệu từ một tập hợp khác sang.
<code>~Set()</code>	Xóa mảng động khi không dùng nữa để dọn bộ nhớ.
<code>operator=</code>	Xóa mảng cũ, chép mảng mới từ tập hợp khác sang.
<code>contains(int x)</code>	Chạy vòng lặp xem số <code>x</code> đã có trong mảng chưa.
<code>insert(int x)</code>	Nếu số <code>x</code> chưa có, tạo mảng to hơn 1 ô rồi nhét <code>x</code> vào cuối.
<code>operator+</code>	Phép Hợp: Gom tất cả các số của hai tập hợp lại với nhau.
<code>operator*</code>	Phép Giao: Chỉ lấy những số mà cả hai tập hợp đều có.
<code>operator-</code>	Phép Hiệu: Lấy các số có ở tập này nhưng không có ở tập kia.
<code>operator[]</code>	Lấy ra con số nằm ở vị trí thứ <code>idx</code> .
<code>operator==</code>	Xem hai tập hợp có chứa các con số y chang nhau không.
<code>operator»</code>	Nhập số lượng và từng con số vào tập hợp.
<code>operator«</code>	In tập hợp ra màn hình theo kiểu { 1; 2; 3 }.

Bảng 2: Bảng các hàm và toán tử của lớp Set

2.2 Cơ sở Giải thuật

2.2.1 Cách quản lý mảng và Thêm số mới (insert)

Thay vì tạo sẵn một mảng khổng lồ, chương trình chỉ mở rộng mảng mỗi khi cần thiết.

1. Trước khi thêm số mới, dùng hàm `contains` để kiểm tra xem số đó đã có chưa (vì tập hợp thì không được trùng số).
2. Nếu chưa có, tạo một mảng tạm to hơn mảng cũ 1 ô.
3. Chép hết các số từ mảng cũ sang mảng tạm, rồi bỏ số mới vào ô cuối cùng.
4. Xóa mảng cũ đi và cho con trỏ `data` nối vào mảng tạm. Tăng biến đếm `size` lên 1.

2.2.2 Giải thuật cho Hợp, Giao, Hiệu

- **Hợp hai tập hợp (+):** Bắt đầu bằng cách lấy toàn bộ tập A. Sau đó chạy vòng lặp lấy từng số trong tập B nhét vào. Hàm `insert` sẽ tự động chặn các số bị trùng.

- **Giao hai tập hợp (*):** Tạo một tập kết quả rỗng. Duyệt qua từng số của tập A, nếu thấy số đó cũng nằm trong tập B (dùng hàm `contains`) thì mới chép vào kết quả.
- **Hiệu hai tập hợp (-):** Tương tự phép Giao, nhưng ngược lại. Duyệt tập A, nếu thấy số nào **không** nằm trong tập B thì mới lấy bỏ vào kết quả.

2.2.3 Giải thuật Nhập dữ liệu an toàn (»)

Khi nhập, chương trình không gán trực tiếp vào mảng mà lại gọi hàm `insert`. Điều này hợp lý vì nếu ta gõ trùng 2 số giống nhau thì hàm `insert` sẽ tự động gạt bỏ số trùng đi, đảm bảo tính chất chuẩn của tập hợp.

2.3 Mã nguồn chương trình

File Mã Nguồn: Set.cpp

```
1 #include <iostream>
2 using namespace std;
3
4 class Set{
5     private:
6         int size;
7         int *data;
8
9     public:
10        Set(int s = 0){
11            size = s;
12            if (size > 0){
13                data = new int[size];
14            }else{
15                size = 0;
16                data = nullptr;
17            }
18        }
19        Set(const Set& other){
20            size = other.size;
21            if (size > 0){
22                data = new int[size];
23                for (int i = 0; i < size; i++){
24                    data[i] = other.data[i];
```

```
25         }
26     }else{
27         data = nullptr;
28     }
29 }
30 ~Set(){
31     delete[] data;
32 }
33 Set& operator=(const Set& other){
34     if (this != &other){
35         delete[] data;
36         size = other.size;
37         if (size > 0){
38             data = new int[size];
39             for (int i = 0; i < size; i++){
40                 data[i] = other.data[i];
41             }
42         }else{
43             data = nullptr;
44         }
45     }
46     return *this;
47 }
48 bool contains(int x) const{
49     for (int i = 0; i < size; i++){
50         if (data[i] == x) return true;
51     }
52     return false;
53 }
54 void insert(int x){
55     if (!contains(x)){
56         int *tmp = new int[size+1];
57         for (int i = 0; i < size; i++){
58             tmp[i] = data[i];
59         }
60         tmp[size] = x;
61         delete[] data;
62         data = tmp;
63         size++;
64     }
```

```
65     }
66     Set operator+(const Set& other) const {
67         Set result = *this;
68         for (int i = 0; i < other.size; i++){
69             result.insert(other.data[i]);
70         }
71         return result;
72     }
73     Set operator*(const Set& other) const{
74         Set result;
75         for (int i = 0; i < size; i++){
76             if (other.contains(data[i])){
77                 result.insert(data[i]);
78             }
79         }
80         return result;
81     }
82     Set operator-(const Set& other) const{
83         Set result;
84         for (int i = 0; i < size; i++){
85             if (!other.contains(data[i])){
86                 result.insert(data[i]);
87             }
88         }
89         return result;
90     }
91     int operator[](int idx) const{
92         if (idx >= 0 && idx < size){
93             return data[idx];
94         }
95         return -1;
96     }
97     bool operator==(const Set& other) const{
98         if (size != other.size) return false;
99         for (int i = 0; i < size; i++){
100             if (!other.contains(data[i])) return false;
101         }
102         return true;
103     }
104     friend istream& operator>>(istream& in, Set& s){
```

```

105         int n, val;
106         cout << "Nhap so luong phan tu cua tap hop: ";
107         in >> n;
108         delete[] s.data;
109         s.size = 0;
110         s.data = nullptr;
111         for (int i = 0; i < n; i++){
112             cout << "Phan tu thu " << i+1 << ": ";
113             in >> val;
114             s.insert(val);
115         }
116         return in;
117     }
118     friend ostream& operator<<(ostream& out, const Set& s){
119         out << "{ ";
120         for (int i = 0; i < s.size; i++){
121             out << s.data[i];
122             if (i < s.size - 1) out << " ";
123         }
124         out << " }";
125         return out;
126     }
127 };
128 int main() {
129     Set A, B;
130     cout << "---- NHAP TAP HOP A ----\n";
131     cin >> A;
132     cout << "\n---- NHAP TAP HOP B ----\n";
133     cin >> B;
134     cout << "\n-----\n";
135     cout << "Tap hop A = " << A << "\n";
136     cout << "Tap hop B = " << B << "\n";
137     Set hop = A + B;
138     cout << "\nHop (A + B)          = " << hop << "\n";
139     Set giao = A * B;
140     cout << "Giao (A * B)              = " << giao << "\n";
141     Set hieu = A - B;
142     cout << "Hieu (A - B)              = " << hieu << "\n";
143     if (A == B) {
144         cout << "\nHai tap hop A va B BANG NHAU.\n";

```

```
145     } else {  
146         cout << "\nHai tap hop A va B KHAC NHAU.\n";  
147     }  
148     return 0;  
149 }
```

2.4 Kết quả chạy thử nghiệm

Thử nghiệm với Tập $A = \{1, 2, 3\}$ và Tập $B = \{2, 3, 4\}$:

Kết quả xuất ra màn hình (Console Output)

```
--- NHAP TAP HOP A ---  
Nhap so luong phan tu cua tap hop: 3  
Phan tu thu 1: 1  
Phan tu thu 2: 2  
Phan tu thu 3: 3  
  
--- NHAP TAP HOP B ---  
Nhap so luong phan tu cua tap hop: 3  
Phan tu thu 1: 2  
Phan tu thu 2: 3  
Phan tu thu 3: 4  
  
-----  
Tap hop A = { 1; 2; 3 }  
Tap hop B = { 2; 3; 4 }  
  
Hop (A + B)          = { 1; 2; 3; 4 }  
Giao (A * B)         = { 2; 3 }  
Hieu (A - B)         = { 1 }  
  
Hai tap hop A va B KHAC NHAU.
```

3 Bài 3: Lớp Ma trận vuông (Matrix)

Yêu cầu đề bài

Cài đặt lớp `Matrix` biểu diễn 01 ma trận vuông cấp n các số thực. Định nghĩa các hàm và toán tử:

- Hàm dựng mặc định, dựng sao chép, hàm hủy.
- Toán tử `>>`, `<<` để nhập, xuất ma trận.
- Toán tử `()` để truy xuất phần tử ở hàng i , cột j .
- Toán tử `+`, `-`, `*` giữa hai ma trận.
- Toán tử gán `=` và toán tử gán kết hợp `+=`.

3.1 Mô tả các thành viên của lớp

3.1.1 Thành phần dữ liệu và Hàm nội bộ (Private)

- `int n`: Kích thước (cấp) của ma trận vuông.
- `double *data`: Con trỏ dùng để tạo mảng động 1 chiều chứa dữ liệu ma trận.
- `allocateMemory()` và `freeMemory()`: Hai hàm tự viết để gom chung công việc tạo mảng và xóa mảng cho gọn code, tránh viết đi viết lại nhiều lần.

3.1.2 Danh sách các hàm và toán tử (Public Methods & Operators)

Phương thức / Toán tử	Chức năng
<code>Matrix(int size = 1)</code>	Tạo ma trận vuông cấp <code>size</code> , mặc định là cấp 1, gán hết bằng 0.
<code>Matrix(const Matrix&)</code>	Chép toàn bộ dữ liệu từ ma trận khác sang.
<code>~Matrix()</code>	Gọi hàm dọn bộ nhớ khi ma trận không dùng nữa.
<code>operator=</code>	Xóa dữ liệu cũ, chép dữ liệu mới từ ma trận khác sang.
<code>operator()</code>	Lấy giá trị của một ô cụ thể tại hàng i , cột j .
<code>operator+, -</code>	Cộng, trừ hai ma trận cùng cấp.
<code>operator*</code>	Nhân hai ma trận vuông theo đúng quy tắc hàng nhân cột.
<code>operator+=</code>	Cộng thêm một ma trận khác vào ma trận hiện tại và lưu luôn.
<code>operator»</code>	Nhập cấp n và từng con số vào ma trận.
<code>operator«</code>	In ma trận ra màn hình cho đẹp, ngay ngắn thành từng cột.

Bảng 3: Bảng các hàm và toán tử của lớp Matrix

3.2 Cơ sở Giải thuật

3.2.1 Sử dụng mảng 1 chiều để biểu diễn ma trận

Thay vì sử dụng con trỏ kép `double **data` (mảng 2 chiều) có thể phát sinh lỗi biên dịch trên một số môi trường IDE như VS Code, chương trình tiếp cận bằng cách sử dụng mảng 1 chiều `double *data` để mô phỏng ma trận 2 chiều.

- Ma trận cấp n sẽ được cấp phát một mảng 1 chiều có kích thước $n \times n$.
- Để truy xuất phần tử tại vị trí hàng i , cột j (tính từ 1), chương trình sử dụng công thức chuyển đổi: `vị trí = (i - 1) * n + (j - 1)`. Cách làm này vừa giúp đảm bảo tính chính xác của dữ liệu, vừa giúp an toàn hơn trong việc quản lý con trỏ.

3.2.2 Quản lý bộ nhớ tập trung

Để tránh việc viết lặp lại các lệnh `new` và `delete[]`, chương trình xây dựng sẵn hai hàm nội bộ (private) là `allocateMemory()` và `freeMemory()`. Việc này giúp mã nguồn ở các hàm dựng, hàm hủy và toán tử gán trở nên gọn gàng, dễ bảo trì và hạn chế tối đa nguy cơ rò rỉ bộ nhớ.

3.2.3 Cài đặt các phép toán (+, -, *, +=)

- **Cộng, trừ:** Chương trình kiểm tra điều kiện hai ma trận cùng cấp trước khi thực hiện. Nếu hợp lệ, vòng lặp sẽ duyệt qua mảng 1 chiều từ 0 đến $n \times n - 1$ để tính toán trực tiếp trên từng phần tử, giúp tối ưu thao tác.
- **Nhân:** Thuật toán áp dụng 3 vòng lặp lồng nhau theo đúng quy tắc hàng nhân cột. Các chỉ số được quy đổi sang tọa độ mảng 1 chiều (tính từ 0) bằng công thức $i * n + j$.

3.3 Mã nguồn chương trình

File Mã Nguồn: Matrix.cpp

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 class Matrix {
6 private:
7     int n;
8     double *data;
9     void allocateMemory() {
10         data = new double[n * n];
11         for (int i = 0; i < n * n; i++) {
12             data[i] = 0;
13         }
14     }
15     void freeMemory() {
16         if (data != nullptr) {
17             delete[] data;
18             data = nullptr;
19         }
20     }
21 public:
22     Matrix(int size = 1) {
23         n = (size > 0) ? size : 1;
24         allocateMemory();
25     }
26     Matrix(const Matrix& other) {
27         n = other.n;
```

```
28     allocateMemory();
29     for (int i = 0; i < n * n; i++) {
30         data[i] = other.data[i];
31     }
32 }
33 ~Matrix() {
34     freeMemory();
35 }
36 Matrix& operator=(const Matrix& other) {
37     if (this != &other) {
38         freeMemory();
39         n = other.n;
40         allocateMemory();
41         for (int i = 0; i < n * n; i++) {
42             data[i] = other.data[i];
43         }
44     }
45     return *this;
46 }
47 double operator()(int i, int j) const {
48     return data[(i - 1) * n + (j - 1)];
49 }
50 Matrix operator+(const Matrix& other) const {
51     if (n != other.n) {
52         cout << "Loi: Hai ma tran khong cung cap!\n";
53         return *this;
54     }
55     Matrix result(n);
56     for (int i = 0; i < n * n; i++) {
57         result.data[i] = data[i] + other.data[i];
58     }
59     return result;
60 }
61 Matrix operator-(const Matrix& other) const {
62     if (n != other.n) {
63         cout << "Loi: Hai ma tran khong cung cap!\n";
64         return *this;
65     }
66     Matrix result(n);
67     for (int i = 0; i < n * n; i++) {
```

```
68         result.data[i] = data[i] - other.data[i];
69     }
70     return result;
71 }
72 Matrix operator*(const Matrix& other) const {
73     if (n != other.n) {
74         cout << "Loi: Hai ma tran khong cung cap!\n";
75         return *this;
76     }
77     Matrix result(n);
78     for (int i = 0; i < n; i++) {
79         for (int j = 0; j < n; j++) {
80             result.data[i * n + j] = 0;
81             for (int k = 0; k < n; k++) {
82                 result.data[i * n + j] += data[i * n + k] * other.
data[k * n + j];
83             }
84         }
85     }
86     return result;
87 }
88 Matrix& operator+=(const Matrix& other) {
89     if (n != other.n) {
90         cout << "Loi: Hai ma tran khong cung cap!\n";
91         return *this;
92     }
93     for (int i = 0; i < n * n; i++) {
94         data[i] += other.data[i];
95     }
96     return *this;
97 }
98 friend istream& operator>>(istream& in, Matrix& m) {
99     cout << "Nhap cap cua ma tran vuong n: ";
100     in >> m.n;
101     m.freeMemory();
102     m.allocateMemory();
103     cout << "Nhap cac phan tu cua ma tran:\n";
104     for (int i = 1; i <= m.n; i++) {
105         for (int j = 1; j <= m.n; j++) {
106             cout << "a[" << i << "][" << j << "] = ";
```

```

107         in >> m.data[(i - 1) * m.n + (j - 1)];
108     }
109 }
110     return in;
111 }
112 friend ostream& operator<<(ostream& out, const Matrix& m) {
113     for (int i = 0; i < m.n; i++) {
114         out << " | ";
115         for (int j = 0; j < m.n; j++) {
116             out << setw(8) << m.data[i * m.n + j] << " ";
117         }
118         out << "|\n";
119     }
120     return out;
121 }
122 };
123 int main() {
124     Matrix A, B;
125     cout << "---- NHAP MA TRAN A ----\n";
126     cin >> A;
127     cout << "\n---- NHAP MA TRAN B ----\n";
128     cin >> B;
129     cout << "Ma tran A:\n" << A << "\n";
130     cout << "Ma tran B:\n" << B << "\n";
131     Matrix sum = A + B;
132     cout << "Tong (A + B):\n" << sum << "\n";
133     Matrix dif = A - B;
134     cout << "Hieu (A - B):\n" << dif << "\n";
135     Matrix pro = A * B;
136     cout << "Tich (A * B):\n" << pro << "\n";
137     int r, c;
138     cout << "Nhap chi so hang r (tu 1): ";
139     cin >> r;
140     cout << "Nhap chi so cot c (tu 1): ";
141     cin >> c;
142     cout << "Gia tri phan tu tai hang " << r << ", cot " << c << " cua
A la: " << A(r, c) << "\n";
143     A += B;
144     cout << "\nMa tran A sau khi thuc hien (A += B):\n" << A << "\n";
145     return 0;

```

146 }

3.4 Kết quả chạy thử nghiệm

Thử nghiệm với hai ma trận vuông cấp 2:

Kết quả xuất ra màn hình (Console Output)

```
--- NHAP MA TRAN A ---
Nhap cap cua ma tran vuong n: 2
Nhap cac phan tu cua ma tran:
a[1][1] = 1
a[1][2] = 2
a[2][1] = 3
a[2][2] = 4

--- NHAP MA TRAN B ---
Nhap cap cua ma tran vuong n: 2
Nhap cac phan tu cua ma tran:
a[1][1] = 2
a[1][2] = 0
a[2][1] = 1
a[2][2] = 2
Ma tran A:
|      1      2 |
|      3      4 |

Ma tran B:
|      2      0 |
|      1      2 |

Tong (A + B):
|      3      2 |
|      4      6 |

Hieu (A - B):
|     -1      2 |
|      2      2 |
```

Tich (A * B):

	4	4	
	10	8	

Nhap chi so hang r (tu 1): 2

Nhap chi so cot c (tu 1): 1

Gia tri phan tu tai hang 2, cot 1 cua A la: 3

Ma tran A sau khi thuc hien (A += B):

	3	2	
	4	6	

—————HẾT—————